



Universidad
Indoamérica

UNITÉ 7 · FONDEMENTS DES SYSTÈMES D'EXPLOITATION

Threads et concurrence

Universidad Tecnológica Indoamérica

Laboratoire TD6

Un thread: une exécution séquentielle à l'intérieur d'un processus

- Aussi « processus léger »: plusieurs exécutions dans un même processus
- Ils partagent tous le **même espace d'adressage**
- **Il n'y a pas de protection mémoire** entre threads
- Un processus = mémoire virtuelle (protection) + threads (exécution parallèle)

Ce qui est partagé, avec de vraies adresses

processus · 8 thread(s)

code

données: shared

0x650ff476602c = 8

heap

descripteurs 0 1 2

partagé par tous les threads

thread 1

pile

local 0x31fee74

registres: PC, SP...

thread 2

pile

local 0x19fbe74

registres: PC, SP...

thread 3

pile

local 0xbffee74

registres: PC, SP...

thread 4

pile

local 0x29fde74

registres: PC, SP...

thread 5

pile

local 0x21fce74

registres: PC, SP...

thread 6

pile

local 0x11fae74

registres: PC, SP...

thread 7

pile

local 0x09f9e74

registres: PC, SP...

thread 8

pile

local 0xb7fde74

registres: PC, SP...

pires espacées de 8,004 Mio (8 Mio + 4 Kio de garde)

Programmes multithreads

- Systèmes embarqués: un programme, des opérations concurrentes
- Le noyau lui-même: des milliers de threads noyau
- Bases de données et serveurs réseau (unité 8)
- Programmes parallèles: OpenMP, un thread par cœur

Aujourd'hui aussi

Navigateurs (threads par onglet et par tâche), moteurs de jeu, inférence de modèles de langage sur CPU (unité 9) et Python, qui depuis la version 3.13 propose une variante sans verrou global (GIL).

Créer un thread coûte moins cher... de combien?

Sortie réelle

```
$ ./create_cost 2000
2000 threads:      41.2 us per pthread_create + pthread_join
2000 processes:   248.8 us per fork + waitpid
a process costs 6.0 times a thread
```

- Aucune page à copier: seulement une nouvelle pile et une structure de tâche
- Le système ordonnance chaque thread: **ne supposez aucun ordre d'exécution**

Créer, terminer et attendre un thread

C

```
int  pthread_create(pthread_t *t, const pthread_attr_t *attr, /* NULL: defaults */
                  void *(*start)(void *), void *arg);
void pthread_exit(void *retval); /* = return retval from start; never returns */
int  pthread_join(pthread_t t, void **retval); /* PTHREAD_CANCELED if t was cancelled */
int  pthread_detach(pthread_t t); /* nobody will join t: freed when it ends */
```

- `attr` choisit, par exemple, *joinable* (par défaut) ou *detached* : on le configure avec `pthread_attr_init` et `pthread_attr_setdetachstate`
- Un thread qui se termine **ne libère pas** sa pile ni son état tant que personne ne le joint (ou s'il est détaché) : sinon, fuite de mémoire, comme un zombie
- `join` suspend l'appelant jusqu'à la fin de `t` et récupère son `retval` ; un thread détaché ne peut pas être joint
- Sous Linux, `pthread_create` utilise l'appel `clone` : même espace d'adressage, mêmes descripteurs

pthread_create, pthread_exit, pthread_join

C tds/td6/threads.c

```
for (int i = 0; i < NTHREADS; i++) {  
    int rc = pthread_create(&tids[i], NULL, start, (void *)(&i));  
    exit_if(rc != 0, "pthread_create"); /* pthread functions return the error code */  
}  
for (int i = 0; i < NTHREADS; i++)  
    exit_if(pthread_join(tids[i], NULL) != 0, "pthread_join");
```

Sortie réelle

```
$ ./threads  
thread 57632 of process 357789: shared[0x650ff476602c] = 1, local[0x7626e31fee74] = 1  
thread 79520 of process 357789: shared[0x650ff476602c] = 4, local[0x7626e19fbe74] = 1  
...
```

Corrections (diapositives 137–140): `-pthread`, et non `-lthread` ; `pthread_*` renvoie 0 ou le numéro d'erreur, **pas** `-l` avec `errno`.

Threads et signaux

Les signaux appartiennent à **tout le processus**. Pour les threads:

PROCESSUS	THREADS
<code>kill()</code>	<code>pthread_kill()</code>
<code>sigprocmask()</code>	<code>pthread_sigmask()</code>
<code>sigsuspend()</code>	<code>sigwait()</code> (un thread dédié à l'attente des signaux)

Concurrence et mémoire partagée: maux de tête

La plupart du temps, les threads touchent des données différentes et l'ordre n'a pas d'importance. Mais avec `x = 1` au départ:

C

```
/* thread 1 */      /* thread 2 */  
x = x + 2;           x = x * 3;
```

Quelles valeurs `x` peut-il prendre? → **non déterministe**.

- C'est vrai aussi bien pour les threads que pour les processus avec mémoire partagée (unité 6)
- Des bogues difficiles à reproduire: ils apparaissent une fois sur des millions d'exécutions
- Un exemple pire (diapositive 146):
`while (x < 1000) x++;` contre
`while (x > -1000) x--;` l'un ou l'autre peut gagner... ou cela peut ne jamais finir

Les 20 entrelacements, un par un

thread 1: $x = x + 2$;

1. $r1 = x$

2. $r1 = r1 + 2$

6. $x = r1$

thread 2: $x = x * 3$;

3. $r2 = x$

4. $r2 = r2 * 3$

5. $x = r2$

mémoire

$x=1$

$x=1$

$x=1$

$x=1$

$x=3$

$x=3$

résultat des 20 entrelacements

$x = 3: 18$

$x = 9: 1$

$x = 5: 1$

réel, TD6 (8 threads, 100 000 billets):

`tickets_race: 103 195 vendus (tickets = -7)`

`tickets_mutex: 100 000 vendus (tickets = 0)`

`tickets_sem: 100 000 vendus (tickets = 0)`

Opérations atomiques

- Elles s'exécutent entièrement ou pas du tout: **indivisibles**
- Typiquement: un accès mémoire de taille native, une instruction simple
- Non atomique: `x++` (lire-ajouter-écrire), accéder à 64 bits sur une machine 32 bits

En C moderne

Depuis C11, deux threads qui accèdent à la même variable sans synchronisation (avec au moins une écriture), c'est une **data race**: **comportement indéfini**. Pour l'atomique:

`_Atomic`, `<stdatomic.h>` ou
`__atomic_add_fetch` (utilisé par `shm_fork`,
unité 6). `volatile` ne rend rien atomique.

Trop de lait: l'exclusion mutuelle en logiciel

THREAD 1	THREAD 2
regarde dans le frigo: pas de lait	
va au magasin	regarde dans le frigo: pas de lait
achète du lait	va au magasin
revient	achète du lait... trop!

- **Synchronisation** : utiliser des opérations atomiques pour coordonner des threads
- **Exclusion mutuelle** : un seul thread à la fois fait l'action
- **Section critique** : le code qu'un seul thread à la fois peut exécuter
- Solutions 1–3 : fermer le frigo à clé, une note, une note par thread... et leurs échecs

Peterson et Dekker: corrects... sur le papier

C

```

/* Peterson, thread 1 */
postit_1 = true;  turn = 2;
while (postit_2 && turn == 2)
    ;                /* busy waiting */
if (!milk) milk++;   /* critical section */
postit_1 = false;

```

- Corrects si les accès mémoire sont atomiques et **ordonnés**
- Mais le compilateur et le processeur **réordonnent** (diapositive 154)
- Et ils font de l'attente active: ils brûlent du CPU

Sortie réelle

```

$ for i in 1 2 3 4 5; do ./peterson; done; for i in 1 2 3 4 5; do ./peterson_fence; done
Peterson no fence   : counter = 4000000, expected 4000000, lost 0
Peterson no fence   : counter = 4000000, expected 4000000, lost 0
Peterson no fence   : counter = 4000000, expected 4000000, lost 0
...
Peterson with fence: counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
...

```

Chaque solution, chaque état

thread 0

```
postit_0 = 1
turn = 1
/* sans barrière */
loop: q = postit_1
if (!q) goto cs
t = turn
if (t ≠ 1) goto cs
goto loop
cs: enter()
r = milk
if (r) goto x
milk++
x: leave()
postit_0 = 0
end
```

thread 1

```
postit_1 = 1
turn = 0
/* sans barrière */
loop: q = postit_0
if (!q) goto cs
t = turn
if (t ≠ 0) goto cs
goto loop
cs: enter()
r = milk
if (r) goto x
milk++
x: leave()
postit_1 = 0
end
```

mémoire partagée

```
milk = 0
postit_0 = 0
postit_1 = 0
turn = 0
```

thread 0 buffer:p0=1, turn=1

thread 1 buffer:p1=1, turn=0

LES DEUX EN SECTION CRITIQUE!

exploration exhaustive

se termine avec 1 litre(s): 14 état(s) final(s)

se termine avec 2 litre(s): 4 état(s) final(s)

contre-exemple le plus court: 12 étapes (étape 12)

Instructions atomiques: test-and-set et compare-and-swap

C

```
int test_and_set(int *p) {    /* atomic */
    int old = *p;  *p = 1;  return old;
}
/* spinlock: busy-waits until the old value is 0 */
void lock(int *l)    { while (test_and_set(l)) ; }
void unlock(int *l) { *l = 0; }
```

- x86: `xchg`, `lock cmpxchg`, `lock xadd` ;
ARM64: `ldxr` / `stxr` ou `cas`
- L'attente active gaspille du CPU et peut provoquer une **inversion de priorité**
- Un verrou sans attente? Non. Sans attente active? En partie →

L'attente active limitée, aujourd'hui: le futex

Sortie réelle

```
$ strace -f -c -e trace=futex ./lock_cost 1 100000; strace -f -c -e trace=futex ./lock_cost 8 100000
1 thread(s) x 100000 lock/unlock: counter = 100000, 48.4 ns per lock/unlock
  0.00    0.000000    0    1    futex
8 thread(s) x 100000 lock/unlock: counter = 800000, 85.5 ns per lock/unlock
100.00    0.223730    27    8013    3116 futex
```

- Mutex libre → une instruction atomique en espace utilisateur, **sans appel système**
- Mutex occupé → `futex(WAIT)` : le noyau endort le thread et le réveille à l'`unlock`
- `pthread_spin_lock` ne dort jamais (attente active): seulement pour des sections très courtes où dormir coûterait plus cher que tourner

Verrous: pthread_mutex_lock / unlock

C tds/td6/tickets_mutex.c

```
static pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

...
for (;;) {
    pthread_mutex_lock(&lock);
    if (tickets ≤ 0) {                               /* checked w.
        pthread_mutex_unlock(&lock);
        break;
    }
    sold++;
    sched_yield();
    tickets--;
    pthread_mutex_unlock(&lock);
}
```

Ou dynamique: pthread_mutex_init(&m, attr)
 (pthread_mutexattr_t ; NULL = valeurs par
 défaut) ... pthread_mutex_destroy(&m) .

Sortie réelle

```
$ ./tickets_race 100000 | tail -3
...
103195 tickets sold out of 100000 (tickets = -7)
```

Sortie réelle

```
$ ./tickets_mutex 100000 | tail -3
...
100000 tickets sold out of 100000 (tickets = 0)
```

Correction (diapositive 162): il renvoie 0 ou le
 numéro d'erreur (EDEADLK seulement pour les
 mutex à vérification d'erreurs), pas -1.

Sémaphores (Dijkstra, fin des années 1960)

Un entier ≥ 0 avec deux opérations atomiques: **P** (*proberen* : attend qu'il soit positif et retire 1) et **V** (*verhogen* : ajoute 1 et réveille).

- Valeur initiale 1 → verrou
- Valeur initiale 0 → attendre un autre thread
- Valeur initiale k → au plus k à la fois

C

```
int sem_init(sem_t *s, int pshared, unsigned value);
int sem_destroy(sem_t *s); /* unnamed */
sem_t *sem_open(const char *name, int flags /* O_CREAT [| O_EXCL] */,
                mode_t mode, unsigned value); /* SEM_FAILED */
int sem_close(sem_t *s); /* named: this process */
int sem_unlink(const char *name); /* named: the name */
int sem_wait(sem_t *s); /* P */
int sem_post(sem_t *s); /* V */
```

Correction (diapositive 167): `sem_open` renvoie `SEM_FAILED` (pas -1) et n'a pas de valeur par défaut: avec `O_CREAT`, il faut donner `value`. Et `sem_close` ne supprime pas le sémaphore: le nom reste dans `/dev/shm` jusqu'à `sem_unlink`.

Le problème du producteur et des consommateurs

- Un producteur écrit des données dans un tampon partagé de taille limitée
- Les consommateurs les retirent
- Le producteur attend si le tampon est plein; le consommateur, s'il est vide
- Le tampon est modifié: il faut une section critique

Exemples: un pipeline de commandes (unité 5), `gcc (cpp | cc1 | as | ld)`, des filtres d'images, des files de messages entre services.

Sortie réelle

```
$ ./prodcons_multi 4 100000
consumer 2: 23918 items
consumer 3: 26423 items
consumer 1: 23037 items
consumer 0: 26622 items
sum = 5000050000, expected 5000050000: OK
```

Deux sémaphores et un mutex, pas à pas

producteur

```
in = (in+1) % BUF
v = 32 · en marche
```

tampon (BUF = 8)



empty = 5 full = 0 out_lock: C2

consommateur 1 · 8

```
wait(full)
attend: full = 0
```

consommateur 2 · 5

```
unlock(out_lock)
en marche
```

consommateur 3 · 9

```
wait(full)
attend: full = 0
```

consommateur 4 · 7

```
lock(out_lock)
attend le mutex
```

les 40 valeurs: vert = une fois · rouge = perdue · orange = répétée



réel, prodcons_multi 4 100000: valeurs par consommateur

C1: 26 622

C2: 23 037

C3: 23 918

C4: 26 423

Variables de condition

C

```
pthread_mutex_lock(&mutex);  
while (!condition)           /* while, not if */  
    pthread_cond_wait(&cond, &mutex);  
/* condition is true, mutex held */  
pthread_mutex_unlock(&mutex);  
/* whoever changes the condition: */  
pthread_cond_signal(&cond);   /* wakes one */  
pthread_cond_broadcast(&cond); /* wakes all */
```

- Attendre n'importe quelle condition (pas seulement un compteur)
- `cond_wait` relâche le mutex et dort de façon **atomique**, puis le reprend au réveil
- `while` et non `if` : il peut y avoir des réveils intempestifs ou un autre thread peut avoir changé la condition

Interblocages

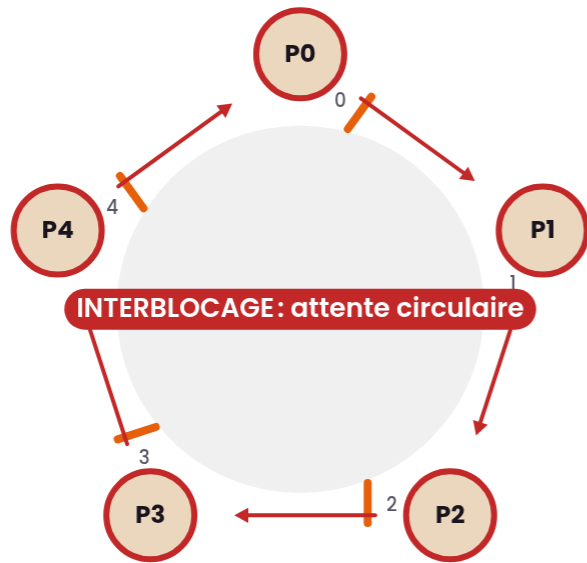
C

```
/* thread 1 */      /* thread 2 */
x.P();              y.P();
y.P();              x.P();  /* ← may never return */
/* ... */           /* ... */
y.V(); x.V();        x.V(); y.V();
```

Cela n'arrive pas toujours: cela dépend de l'exécution. Il faut plus d'une ressource et cela ne se résout pas sans intervention extérieure.

- Ressources: CPU, mémoire, disque, périphériques; partageables ou non; réquisitionnables ou non
- Exemple: un carrefour avec priorité, chaque voiture tenant un tronçon et attendant le suivant
- Issues: revenir en arrière, retirer une ressource (réquisition)... avec un risque de famine

Le dîner des philosophes



- pense
- a faim
- mange
- fini
- attend (graphe)

Conditions de Coffman

- ✓ exclusion mutuelle
- ✓ détention et attente
- ✓ pas de réquisition
- ✓ attente circulaire

Les quatre conditions et comment les prévenir

1. Exclusion mutuelle
 2. Détenir des ressources en attendant d'autres
 3. Pas de réquisition
 4. Attente circulaire: T_1 attend T_2 , ..., T_n attend T_1
- **Ordre global** d'acquisition (rompt la 4): la correction du TD6
 - Réquisitionner (rompt la 3), avec un risque d'état incohérent
 - Détecter les cycles dans le graphe d'allocation; les éviter avec l'algorithme du banquier
 - Délais (`trylock`, `timedlock`): aucune garantie

Conclusion

- Problèmes: résultats non déterministes, interblocages, famine
- Toute écriture sur une ressource partagée → dans une section critique
- L'exclusion mutuelle en logiciel marche sur le papier; sur du vrai matériel, **mutex, sémaphores et atomiques**
- Les interblocages sont difficiles à reproduire et exigent les quatre conditions

TD6 corrigé + défi IA

- threads, tickets_race/mutex/sem
- prodcons, prodcons_multi, philosophers (et sa version qui s'interbloque)
- Nouveau: create_cost, peterson, lock_cost

Bash

```
make -C tds && tds/tests/run.sh td6
```



Universidad
Indoamérica

À RETENIR

Trois questions

1. Pourquoi `tickets_race` vend-il plus de billets qu'il n'y en a ?
2. Pourquoi Peterson échoue-t-il sur un x86 et qu'est-ce qui le corrige ?
3. Quelle condition l'ordre global des baguettes rompt-il ?